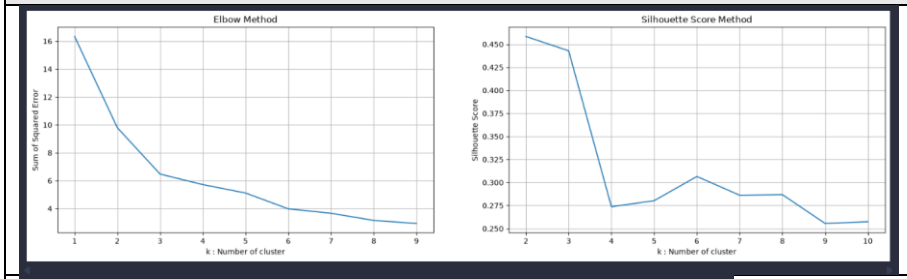
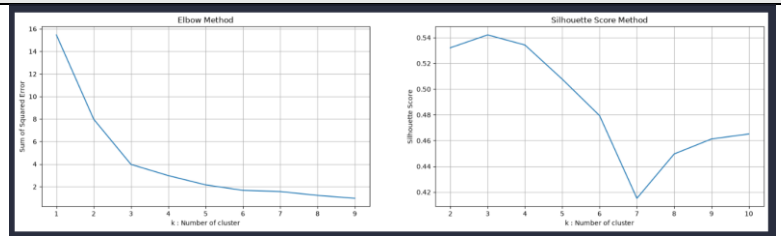
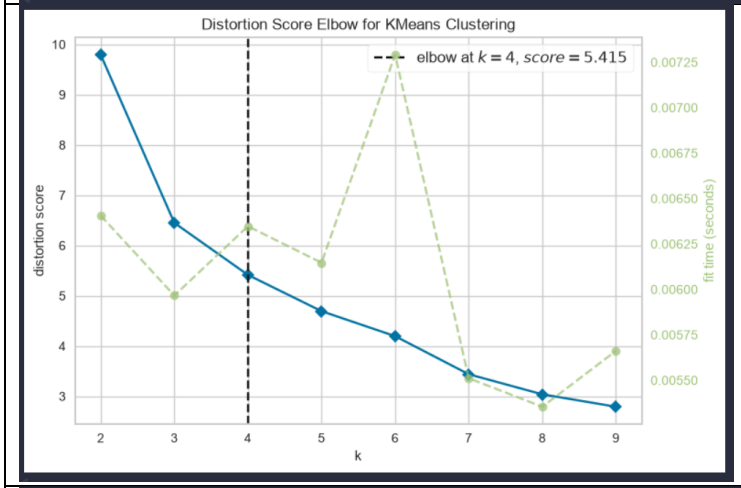
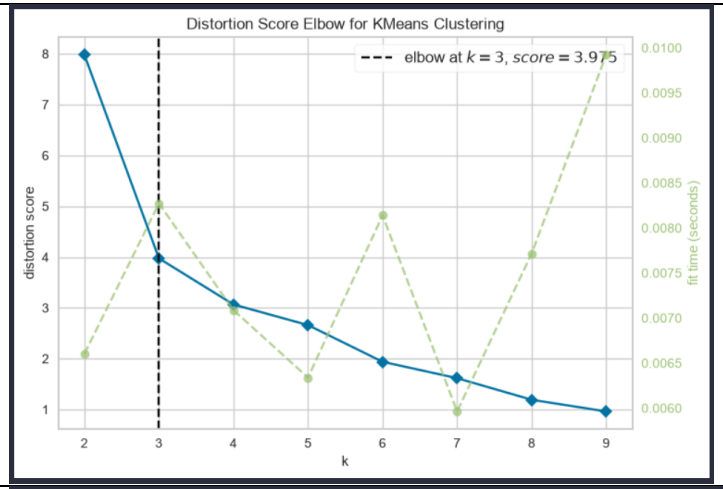
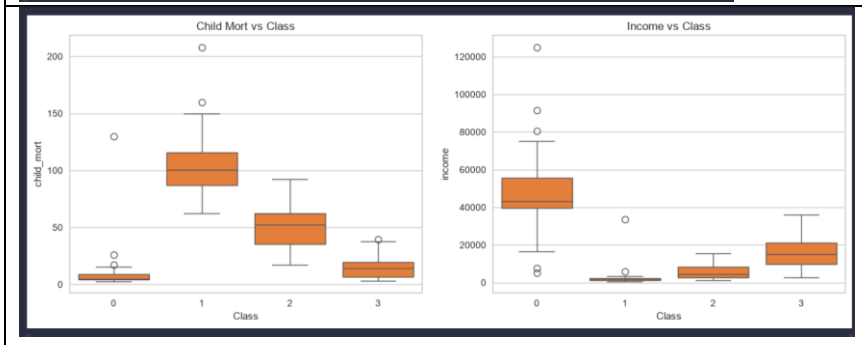
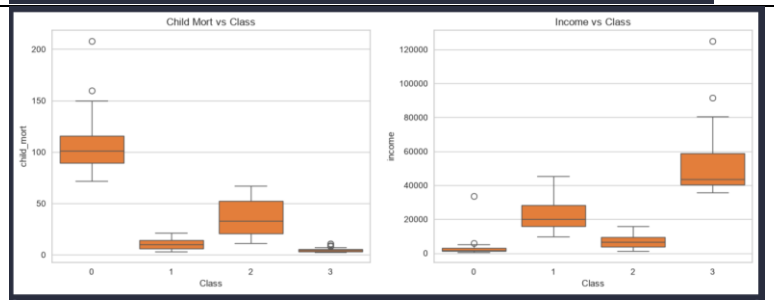
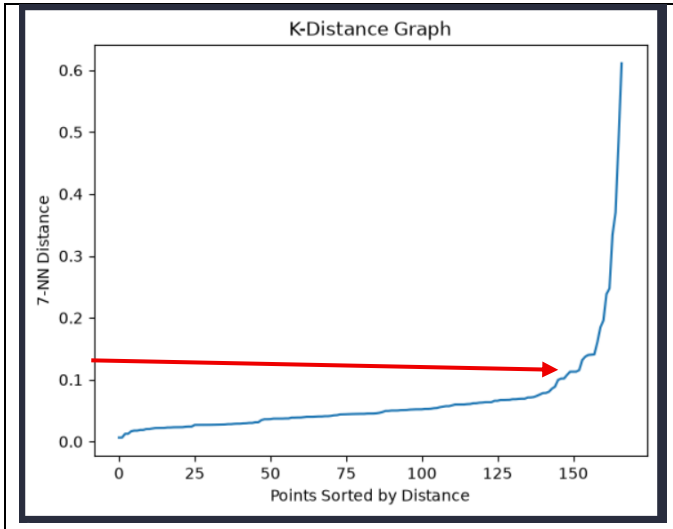
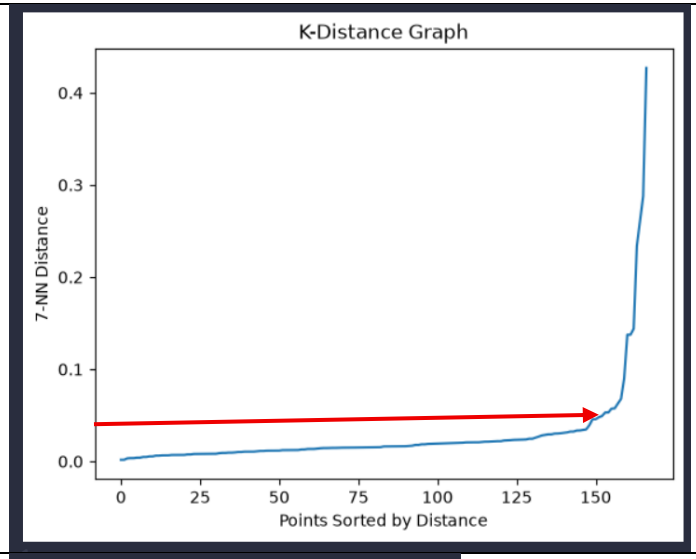
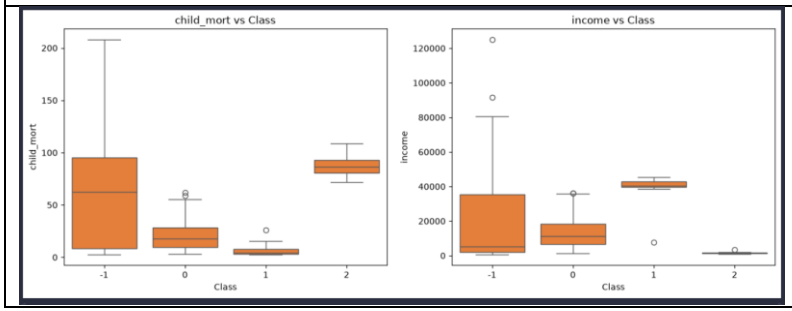
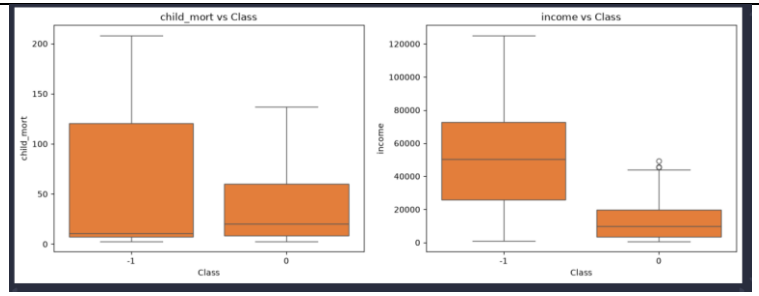
<table>
<tr><th>With all features:</th><th>With feature selection</th><th>Conclusion</th></tr>
<tr><td>

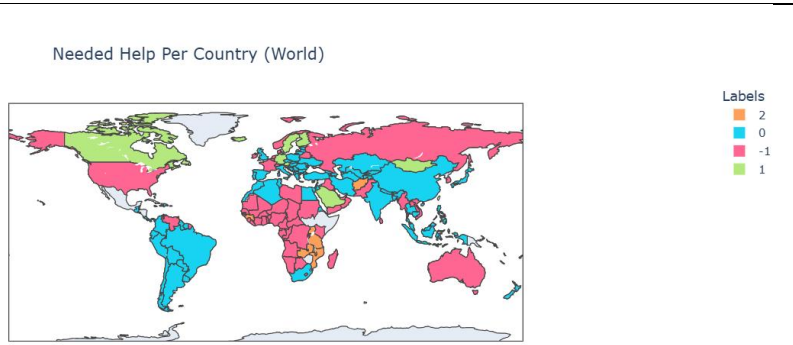
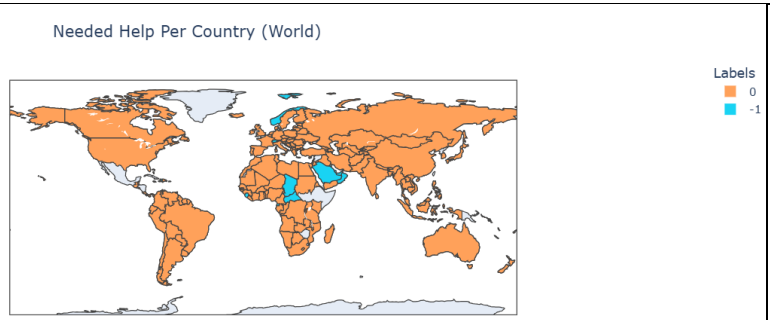
```
Categorical Features : country
Numerical Features : child_mort exports health imports income inflation life_expec total_fer gdpp
```

</td><td>

```
Categorical Features : country
Numerical Features : gdpp child_mort income
```

</td><td></td></tr>
<tr><td colspan="3">KMEANS</td></tr>
<tr><td> 
</td><td> 
</td><td>Datapoints in feature selection are closer to each other (silhouette score is higher) so we have less noise</td></tr>
<tr><td> 
</td><td> 
</td><td>We need less clusters with feature selection</td></tr>
<tr><td> 
</td><td> 
</td><td>The result of the box plots are almost the samen. (Keep in mind that the labels are random, 0 label on the all features data is NOT the same as 0 label on the feature selection data.</td></tr>
<tr><td colspan="3">DBSCAN</td></tr>
</table>

		All points in the dataset are close to each other until 0.12 on all feature data, but for feature selection it stays close until +-0.04 so we have to change things in the code for DBSCAN to work properly
Number of Clusters : 3 Number of Outliers : 67	Number of Clusters : 1 Number of Outliers : 16	We see that the model doesn't see the clusters anymore. With feature selection we only have ONE! Cluster left, this means this is not a good approach for this model.
		useless

<table>
<tr>
<td> 
</td>
<td> 
</td>
<td>useless</td>
</tr>
<tr>
<td colspan="3">Lets change the approach for feature selection on DBSCAN</td>
</tr>
<tr>
<td></td>
<td>

```
#db = DBSCAN(eps = 0.080, min_samples = 8).fit(df_no_country)
db = DBSCAN(eps = 0.050, min_samples = 8).fit(df_no_country)
core_samples_mask = np.zeros_like(db.labels_, dtype=bool)
core_samples_mask[db.core_sample_indices_] = True
labels = db.labels_

# Number of clusters in labels, ignoring noise if present

n_clusters_ = len(set(labels)) - (1 if -1 in labels else 0)
n_noise_ = list(labels).count(-1)
print('Number of Clusters : ', n_clusters_)
print('Number of Outliers : ', n_noise_)

data['Class'] = labels; compressed_data['Class'] = labels
```

[22] ✓ 0.0s

```
Number of Clusters :  2
Number of Outliers :  37
```

</td>
<td>We change the eps (distance between neighbours)
Now we have 2 clusters, this is already way better.</td>
</tr>
<tr>
<td></td>
<td>

```
#db = DBSCAN(eps = 0.080, min_samples = 8).fit(df_no_country)
db = DBSCAN(eps = 0.042, min_samples = 8).fit(df_no_country)
core_samples_mask = np.zeros_like(db.labels_, dtype=bool)
core_samples_mask[db.core_sample_indices_] = True
labels = db.labels_

# Number of clusters in labels, ignoring noise if present

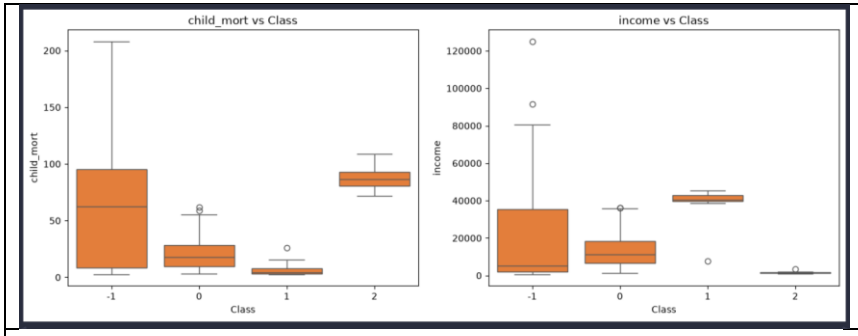
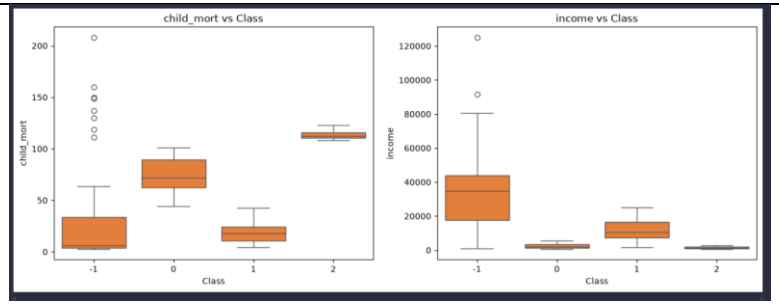
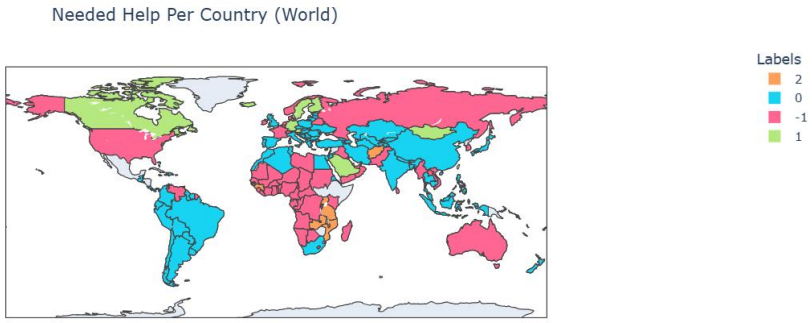
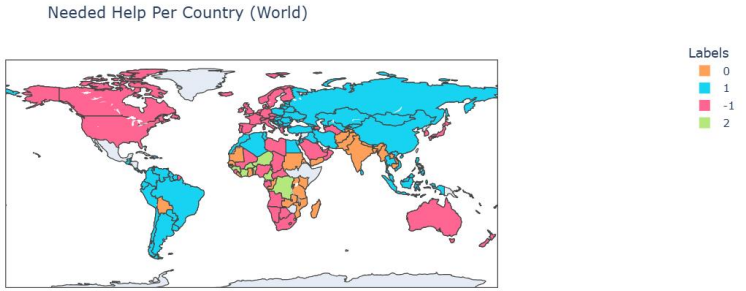
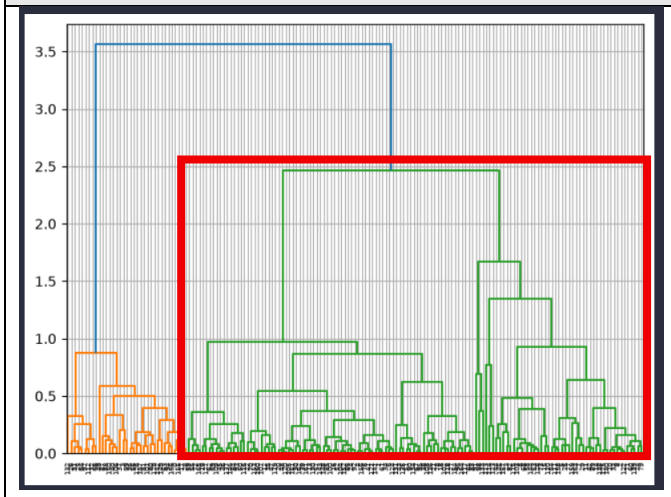
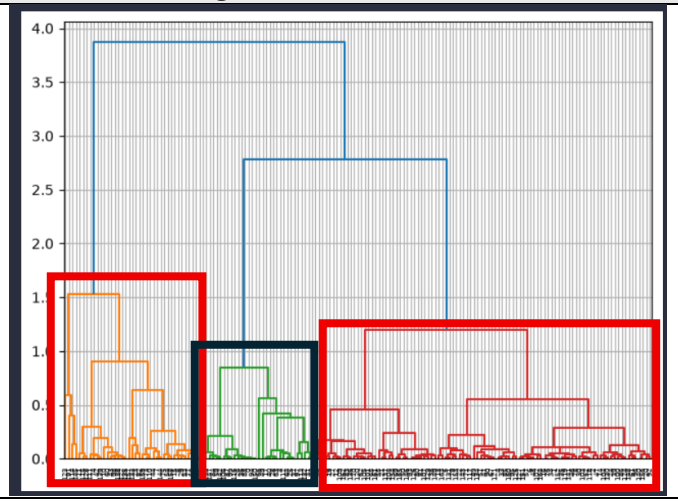
n_clusters_ = len(set(labels)) - (1 if -1 in labels else 0)
n_noise_ = list(labels).count(-1)
print('Number of Clusters : ', n_clusters_)
print('Number of Outliers : ', n_noise_)

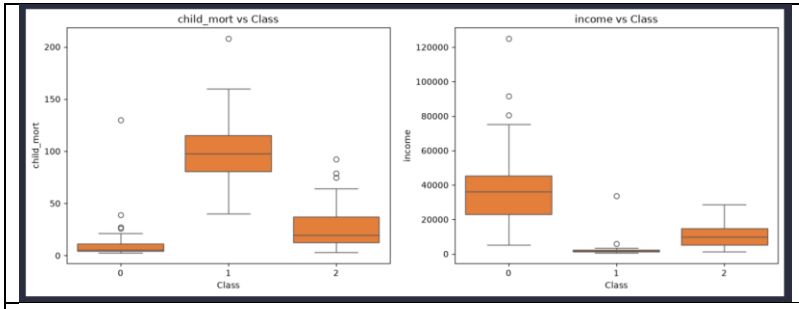
data['Class'] = labels; compressed_data['Class'] = labels
```

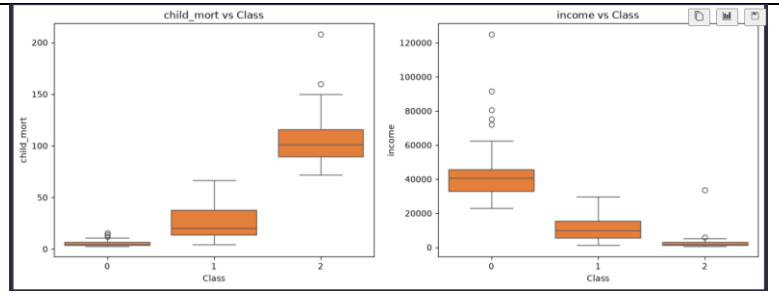
1] ✓ 0.0s

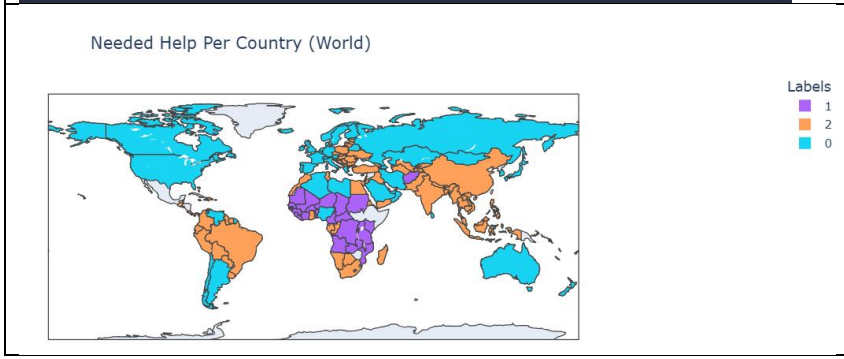
```
Number of Clusters :  3
Number of Outliers :  54
```

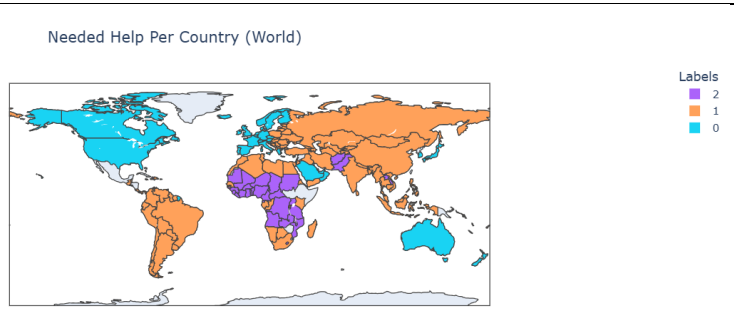
</td>
<td>I tried a few more eps and now we have the same amount of clusters BUT less outliers than when we used all features</td>
</tr>
</table>

<table>
<tr>
<td> 
</td>
<td> 
</td>
<td></td>
</tr>
<tr>
<td> 
</td>
<td> 
</td>
<td></td>
</tr>
<tr>
<td colspan="3">Hierarchical clustering</td>
</tr>
<tr>
<td>  </td>
<td>  </td>
<td>The groups are more clustered, we can see a clearer difference between clusters when we do feature selection.</td>
</tr>
</table>


child_mort vs Class
income vs Class
child_mort
income
Class


child_mort vs Class
income vs Class
child_mort
income
Class


Needed Help Per Country (World)
Labels
1
2
0


Needed Help Per Country (World)
Labels
2
1
0